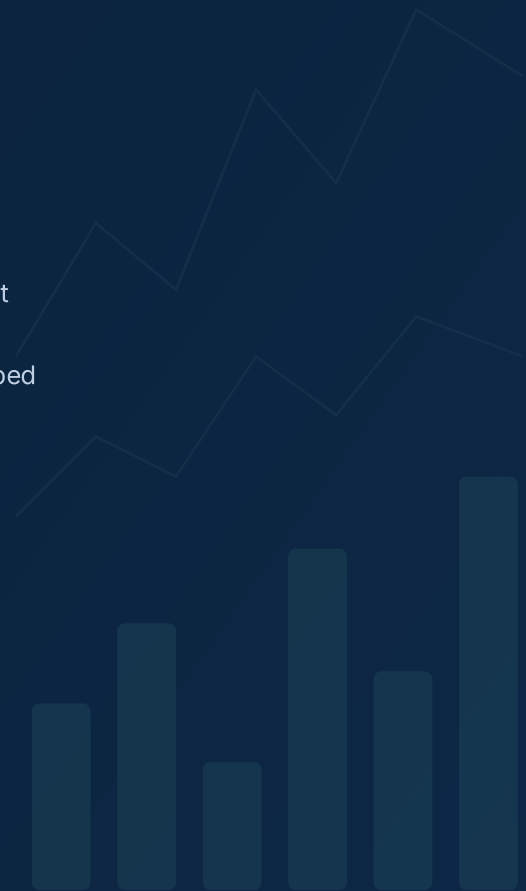


TerraTrac Insights

Demand Forecasting & Supply Planning

A 12-week demand forecast, a safety-stock and reorder-point plan, and a supplier-risk assessment built from two years of weekly sell-through and 220 actual purchase orders — shipped as a live, published dashboard.

**25**

SKUS

4

DISTRIBUTION CENTERS

104 wks

DEMAND HISTORY

220

PURCHASE ORDERS

Tahtabir Rahman

GitHub: @tahtabirrahman

Repository github.com/tahtabirrahman/terratrac-insightsLive dashboard trac-planner-insight.lovable.app

What This Project Is

A self-directed data analytics case study: I scoped the analytical problem from a synthetic supply-chain dataset, built a working demand-forecasting and inventory-planning product from it, and shipped it as a live, published dashboard.

PROJECT

TerraTrac Insights — Demand Forecasting & Supply Planning

A fictional distributor of heavy-construction spare parts across five categories, four regional DCs and five category-specialist suppliers with different lead times and reliability.

OBJECTIVE

Turn raw sell-through into a plan a leadership team can act on

Build a 12-week forecast, a safety-stock and reorder-point plan, and a supplier-risk assessment grounded in actual purchase-order history rather than stated supplier claims.

DATA

Six linked tables in one workbook — 10,400 weekly demand rows, 25 SKUs, 4 DCs, 220 purchase orders.

TOOLS

Python (pandas, NumPy, SciPy/statsmodels), Lovable for the dashboard product, Git/GitHub.

OUTCOME

A published, interactive dashboard: forecast accuracy, inventory risk flags, replenishment plan, supplier scorecard.

Problem statement

TerraTrac's planning process had no formal reorder-point discipline. Nobody could answer, with evidence: which SKU × warehouse positions are actually at stockout risk right now, which suppliers' reliability claims hold up against real delivery data, and whether current demand forecasts are usable or just noise.

Key questions the analysis answers

01

Which positions are below reorder point today, and how urgently?

02

How accurate is a trend + seasonality + promotion forecast on a blind holdout?

03

Which supplier carries the most lead-time risk — and does the data back the stated claims?

04

What should be reordered this cycle, respecting MOQ and pack size?

Why This Problem Matters

Heavy-equipment parts distribution runs on thin margins for error: a stocked-out hydraulic seal or undercarriage part can idle a customer's construction equipment, and lead times of 2–5 weeks mean a planning mistake isn't quickly correctable. Two failure modes are common in under-instrumented supply operations — and both show up in this dataset.

Silent under-stocking

Inventory drifts below what's needed to cover a lead time, and nobody notices until a stockout actually happens, because there's no reorder-point discipline flagging it in advance.

Unverified supplier claims

Reliability figures used for planning come from supplier scorecards that are rarely checked against what the purchase-order data actually shows.

The analytical path



Objectives

- 01

Produce a defensible 12-week demand forecast per SKU, correctly handling known data issues — promotions and a stockout-censored demand period — rather than ignoring them.
- 02

Convert that forecast into a safety-stock and reorder-point plan with a stated service-level assumption.
- 03

Flag stockout-risk and overstock positions, and produce a replenishment recommendation that respects MOQ and pack size.
- 04

Score supplier lead-time and reliability risk using actual purchase-order history, not the stated figures.

Business questions this analysis answers

QUESTION	WHERE IT IS ANSWERED
Which positions need action this week?	Supply Planning tab — reorder point, safety stock and a risk flag per SKU × warehouse.
Can we trust the forecast?	Reported honestly against a naive baseline, on an out-of-sample holdout the model never saw.
Which supplier is the real risk?	Supplier Risk tab — lead time and on-time rate computed from 220 actual purchase orders.

The Data Behind the Analysis

A synthetic case-study dataset — six linked tables in one workbook, deliberately built with realistic messiness rather than clean, ready-to-plot data.

DEMAND HISTORY
ROWS
10,400
 25 SKUs × 4 DCs × 104 weeks

SKUS
25
 5 categories, 3 ABC classes

PURCHASE ORDERS
220
 across 5 suppliers

HISTORY SPAN
104 wks
 ~2 years, weekly

Tables

TABLE	WHAT IT HOLDS
Product_Master	25 SKUs — category, ABC class, unit cost/price, MOQ, pack size, shelf life
Warehouse_Master	4 distribution centers — region, storage capacity
Supplier_Master	5 suppliers — stated average lead time, lead-time variability, OTIF reliability
Demand_History	10,400 rows — weekly sell-through by SKU × warehouse, with promotion and stockout flags
Inventory_Snapshot	Current on-hand and on-order quantity by SKU × warehouse
Purchase_Order_History	220 orders — promised vs. actual delivery date, used for real lead-time and reliability analysis

Six tables, one grain that matters: **SKU × warehouse × week**. Everything downstream — forecast, safety stock, replenishment — is computed at that grain and rolled up from it.

Core Table, Known Issues, Stated Assumptions

Demand_History is the table the whole analysis rests on — and it carries the dataset's deliberate data-quality problems.

Data dictionary — Demand_History

COLUMN	TYPE	NOTES
Week_Start_Date	Date	Weekly bucket
SKU_ID / Warehouse_ID	Text	Foreign keys to Product_Master / Warehouse_Master
Units_Sold	Integer	Weekly sell-through
Unit_Price_USD	Decimal	Price that week
Promotion_Flag	Boolean	10 promotional weeks, causing real demand spikes
Stockout_Flag	Boolean	Marks a 6-week window where observed demand is artificially suppressed by a supply disruption — a data-quality issue that has to be corrected, not ignored

Known data-quality issues (by design)

- Each SKU has its own trend, seasonal cycle and noise level — no uniform pattern to exploit.
- A 6-week supply disruption (weeks 41–46) suppresses observed Undercarriage demand at the two Asia-Pacific DCs.
- On-hand inventory is noisy relative to recent demand — some positions overstocked, others genuinely at risk.
- Supplier lead times vary in both mean and standard deviation.

STATED ASSUMPTIONS

Service level: 98% for ABC Class A SKUs, 95% for B/C — a judgment call, not given in the data, stated explicitly rather than left implicit.

Replenishment review cycle: 4 weeks.

What Was Actually Used

Listed honestly, including how each tool was actually used — not a generic tools list.

ANALYSIS

Python — pandas, NumPy, SciPy / statsmodels

Used to build the analysis pipeline: demand forecasting (trend + seasonality + promotion regression), stockout-censoring correction, safety-stock / reorder-point calculation, and supplier-risk scoring from raw purchase-order data. The initial pipeline was AI-drafted from a specification; I then verified its output against the source data by hand, found two real calculation bugs (page 10), traced each to its exact line, corrected it, and re-verified before publishing.

PRODUCT

Lovable — AI application builder

Used to generate the interactive dashboard (React + TypeScript + Tailwind) from a written specification, wired to the analysis output as a static JSON data file. I directed the build, requested specific fixes and layout changes through it, and managed the publish / deploy cycle.

VERSION CONTROL

Git & GitHub

Repository hosting, commit-level data fixes, file organization, and public deployment of the corrected dashboard and analysis script.

How the stack maps to the work

PANDAS / NUMPY

Data shaping, joins, weekly aggregation

SCIPY / STATSMODELS

Regression fit, service-level Z factors

LOVABLE / REACT

Published interactive dashboard

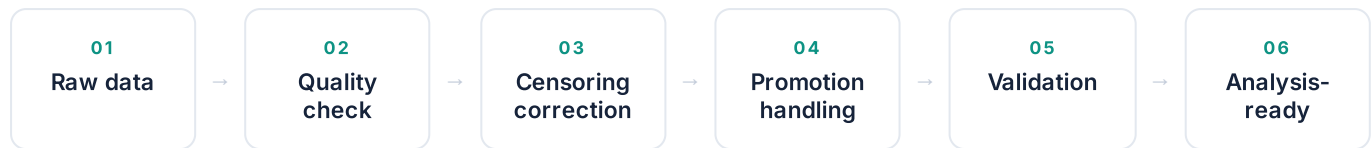
GIT / GITHUB

Versioned fixes, public repository

Not used in this project. SQL · Power BI · Excel-based analysis · Jupyter Notebook · Tableau — not part of this project's actual toolchain, so no fabricated section for any of them appears in this document.

Handling the Censored-Demand Problem

The dataset's central data-quality challenge: a real 6-week supply disruption suppressed observed demand for Undercarriage SKUs at the two Asia-Pacific DCs. Reported sell-through in that window understates true demand — it is censored, not soft.



The three options, and why I picked one

OPTION	PROBLEM WITH IT
Ignore it	Forecast and safety stock both come out too low for the SKUs and warehouses that need the buffer most.
Exclude those weeks entirely	Shrinks the training sample and biases the seasonal estimate for that calendar period.
Correct it (chosen)	Reconstructs a defensible estimate of true demand without discarding data.

Method — sibling-warehouse ratio correction

For each affected SKU, I computed the average ratio of that Asia-Pacific DC's demand to the combined demand at the two unaffected DCs (Dallas, Rotterdam), using only clean, non-promotional, non-censored weeks as the basis. That ratio was then applied to the unaffected DCs' demand during the censored window to estimate what the affected DC would have sold without the disruption.

Result: measured Undercarriage demand during the censored weeks was suppressed by an average of **163%** relative to the corrected estimate. Left uncorrected, this would have understated both the forecast and the safety-stock requirement for the highest-risk product category in the portfolio.

Promotion handling

10 known promotional weeks cause genuine demand spikes. Rather than let them distort the trend/seasonal fit, I excluded them from the baseline fitting step and estimated a separate promotion-uplift multiplier from those weeks specifically.

Data validation

Cross-checked corrected demand totals, the stockout-flag date range and supplier reliability figures against the raw source tables rather than trusting intermediate output at face value — this is what surfaced the two calculation bugs on page 10.

No missing-value imputation or deduplication was required — the source tables were structurally complete. The data-quality issues here are about correctness of interpretation (censored demand, unverified supplier claims), not incomplete records.

Demand Patterns and the Forecasting Approach

Each SKU carries its own trend, seasonal cycle and noise level — there is no single pattern that fits all 25. Below is the network-total weekly history for two SKUs in scope, with the 12-week forecast and the blind holdout window marked.



Weekly network-total demand with the 12-week forecast; the shaded band is the 8-week blind holdout window.

WMAPE, 8-WEEK BLIND
HOLDOUT

23.1%

model, not trained on this window

NAIVE BASELINE (52-WK AVG)

24.0%

what it is compared against

HOLDOUT LENGTH

8 wks

true out-of-sample test

Method

Trend × seasonality × promotion-uplift regression, fit per SKU × warehouse on non-promotional weeks. Chosen over ARIMA/ETS given the data: 104 weeks is barely two full seasonal cycles, and each SKU's noise profile differs enough that a transparent, auditable model is more defensible to a planning audience than a marginally-tunable black box.

Key takeaway. The model is reported honestly against a naive baseline on data it never trained on — not measured on the same data it was fit to, which would overstate accuracy.

Core Calculations

Two representative pieces of the analysis pipeline — not every line, just what demonstrates the actual analytical logic.

Safety stock & reorder point

Accounts for both demand variability and supplier lead-time variability — not demand variability alone, which is the more common and less complete version of this formula.

```
safety_stock = Z * sqrt(
    lead_time_weeks * demand_std**2
    + avg_weekly_demand**2 * lead_time_std_weeks**2
)
reorder_point = avg_weekly_demand * lead_time_weeks + safety_stock

# Z = 2.05 (~98% service) for ABC Class A, 1.65 (~95%) for B/C
# lead_time_weeks / lead_time_std come from actual PO history, not the supplier's stated
average
```

Purpose: converts the forecast into an actionable trigger level per SKU × warehouse, with the service-level assumption stated rather than buried.

Stockout-censoring correction

The sibling-warehouse ratio method described on page 07, as implemented.

```
# true censored weeks, from the Stockout_Flag column directly
censored_weeks = demand_history.loc[demand_history.Stockout_Flag == 1,
'Week_Start_Date'].unique()

sibling_avg = (demand_NA1 + demand_EU1) / 2
clean_weeks = ~(is_censored | is_promotion)
ratio = (demand_AP[clean_weeks] / sibling_avg[clean_weeks]).mean()

corrected_demand = ratio * sibling_avg[censored_weeks]
```

Purpose: restores an estimate of true demand for the affected weeks so the forecast and safety stock are not trained on artificially suppressed sales.

Why this matters more than the code itself. The formula and the correction method are both defensible on their own. What made this a real analytical exercise wasn't writing this code — it was verifying that the actual pipeline, AI-drafted from a specification, implemented them correctly. It didn't, in two places. That is the subject of the next page.

Two Bugs Found by Checking the Output Against the Data

The part of the project that actually demonstrates analytical judgment: not writing code, but not trusting it either — checking derived numbers against source data and catching what didn't add up.

Bug 01 — four suppliers with an “identical” 70% on-time rate

The supplier scorecard showed Trackline, Circuit & Cab, Vantage and Ironclad all at exactly 70% on-time delivery, with Meridian at 60%. Four independently-measured suppliers landing on the same figure is not a coincidence — it is a rounding artifact. Traced to this:

```
- sup_actual = po.groupby("Supplier_ID").agg(  
-     OnTime_Pct_Actual=("On_Time_Flag", "mean"),  
- ).round(1)                                # rounds the 0-1 FRACTION to 1 decimal first  
- sup_actual["OnTime_Pct_Actual"] *= 100    # then multiplies – precision already lost  
  
+ sup_actual = po.groupby("Supplier_ID").agg(  
+     OnTime_Pct_Actual=("On_Time_Flag", "mean"),  
+ )                                          # no premature rounding  
+ sup_actual["OnTime_Pct_Actual"] = (sup_actual["OnTime_Pct_Actual"] * 100).round(1)
```

Rounding a 0–1 fraction to one decimal before multiplying by 100 collapses any value into a 10-point bucket — 0.583, 0.651, 0.660 and 0.683 all round to 0.7. Fixing the order of operations recovered the real, distinct figures: **65.1%, 58.3%, 66.0%, 68.3%, 73.7%**.

Bug 02 — an off-by-one week range in the stockout correction

The censored-week correction used a hardcoded date range instead of reading it from the data's own Stockout_Flag column:

```
- CENSORED_WEEKS = range(40, 46)    # weeks 40-45 – one week early  
  
+ CENSORED_WEEKS = range(41, 47)    # weeks 41-46 – matches Stockout_Flag == 1
```

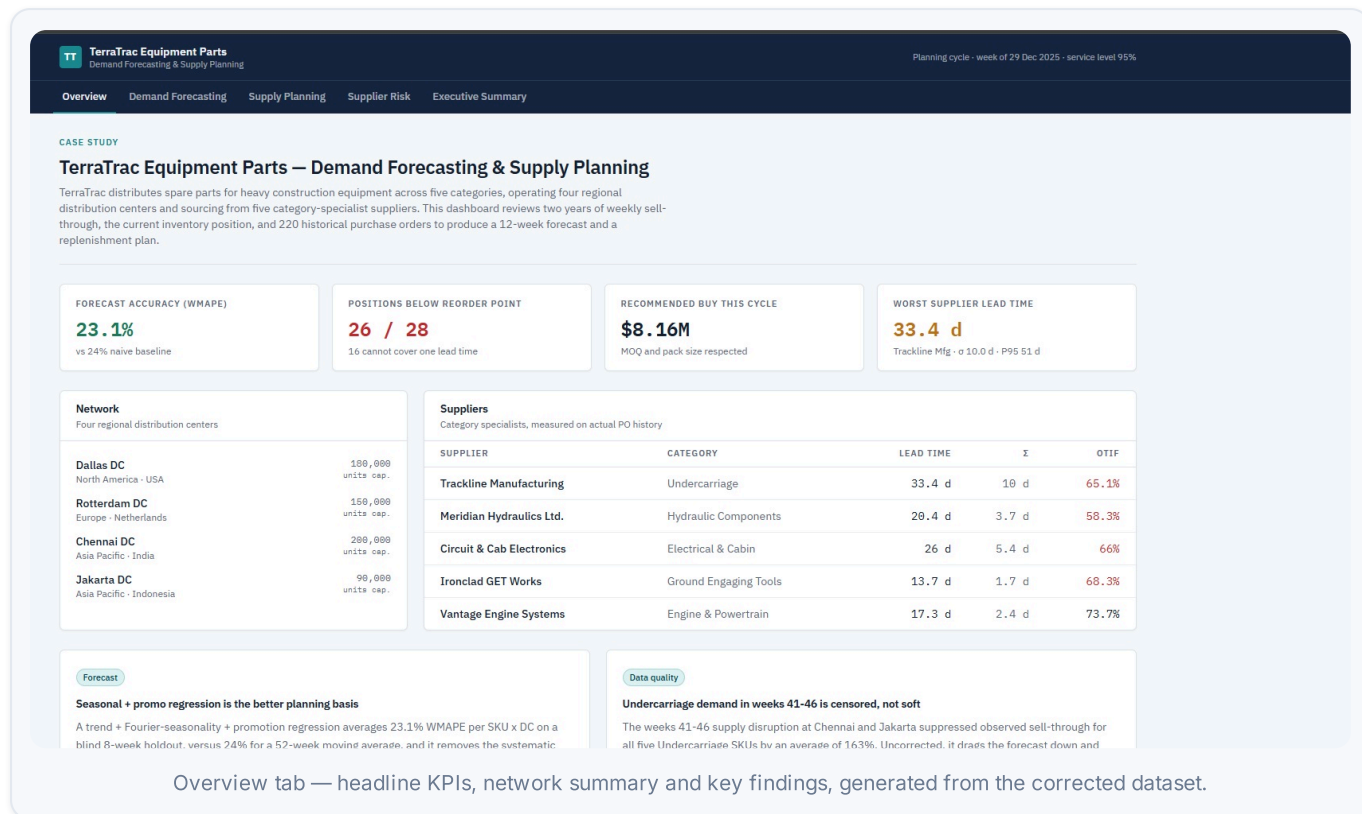
This silently corrected one week that was never censored, and missed the real sixth censored week entirely — wrong on both ends of the range.

What this demonstrates

Both bugs were caught the same way: by treating a generated number as a claim to verify, not a fact to accept — cross-checking supplier figures against each other (four identical values is implausible) and checking a date range against the column that actually defines it, rather than a hardcoded assumption.

The Published Product

Live, interactive dashboard built on the corrected analysis output. Screenshots are from the actual deployed site, not mockups — trac-planner-insight.lovable.app



WHAT IT SHOWS

Forecast accuracy, positions below reorder point, recommended buy value and worst supplier lead time, at a glance.

WHY IT MATTERS

A leadership audience needs the headline numbers before the detail.

Main insight. 26 of 28 monitored positions sit below their reorder point — not a handful of isolated stockouts, a structural pattern. **Business implication:** replenishment discipline needs to change, not just this cycle's order quantities.

What the Analysis Found

01

26 of 28 monitored SKU × warehouse positions are below their reorder point

16 of those cannot even cover a single replenishment lead time from on-hand stock alone. This isn't concentrated in one or two problem SKUs — it is network-wide, pointing to a missing reorder-point process rather than isolated bad luck.

02

Trackline Manufacturing is a structural risk, not just a slow supplier

33.4-day average actual lead time with a 10-day standard deviation — more than 3× the variability of the most consistent supplier — and the sole source for the Undercarriage category. It also caused the demand-censoring event that had to be corrected in the forecast.

03

Every supplier's real on-time rate is materially worse than its stated reliability figure

A gap of 12–33 percentage points across all five suppliers, verified against 220 actual purchase orders. This isn't a Trackline-specific issue — it suggests the stated reliability field shouldn't be trusted for planning without independent verification, for any supplier.

04

A seasonality + promotion-aware forecast beats a naive baseline, honestly measured

23.1% WMAPE vs. 24.0% for a 52-week moving average, on an 8-week blind holdout. The margin is real but not dramatic — reported as such rather than overstated.

05

Correcting censored demand changed the picture for the highest-risk category

The Undercarriage stockout-censoring correction raised measured demand during the affected weeks by 163%. Left uncorrected, both the forecast and the safety-stock buffer for the category with the least reliable supplier would have been understated — compounding the risk instead of catching it.

What TerraTrac Should Do With This

Each recommendation traces back to a specific finding: insight → recommendation → expected business impact.

THIS WEEK

Release the catch-up replenishment plan

INSIGHT — 16 positions cannot cover one lead time from on-hand stock.

RECOMMENDATION — place the \$8.16M replenishment plan across the 7 monitored SKUs now, respecting MOQ and pack size, and expedite the 16 critical positions ahead of the routine cycle.

EXPECTED IMPACT — closes the most immediate stockout exposure before it becomes a customer-facing outage.

NEXT PLANNING CYCLE

Adopt the seasonality + promotion forecast as the planning basis

INSIGHT — it beats the current naive-average approach by a real, if modest, margin on a blind holdout.

RECOMMENDATION — replace the 52-week moving average with this model, and track WMAPE and bias per SKU going forward rather than assuming it stays accurate.

EXPECTED IMPACT — reduces both over- and under-forecasting, particularly around known promotional weeks.

30 DAYS

Re-parameterize safety stock for Undercarriage specifically

INSIGHT — Trackline's lead-time variability is the highest in the network by a wide margin.

RECOMMENDATION — raise the service-level target for Undercarriage SKUs above the network-wide 95%, and size safety stock against a P95 lead time rather than the average.

EXPECTED IMPACT — reduces stockout exposure in the highest-risk category without over-buffering the rest of the portfolio.

90 DAYS

Qualify a secondary Undercarriage source

INSIGHT — Trackline is sole-sourced for the category and carries the network's highest lead-time risk.

RECOMMENDATION — open a secondary-sourcing conversation, and use the verified on-time rate — not the supplier's own stated figure — in the next supplier review.

EXPECTED IMPACT — reduces single-source exposure for the category most likely to cause a stockout.

What Was Actually Achieved

I scoped a supply-chain analytics problem, directed AI tools to build both the analysis pipeline (Python) and the product (an interactive dashboard via Lovable), and then did the part that's easy to skip: treated that output as a first draft to verify, not a finished answer to publish as-is.

FORECAST ACCURACY

23.1%

WMAPE on an 8-week blind holdout, vs 24.0% naive

POSITIONS BELOW REORDER POINT

26 / 28

16 cannot cover one lead time

REPLENISHMENT PLAN VALUE

\$8.16M

MOQ and pack size respected

Where the real work happened

Cross-checking the supplier scorecard surfaced four suppliers reporting an identical, statistically implausible 70% on-time rate. Tracing it led to a one-line rounding-order bug. Checking the stockout-correction logic against the raw Stockout_Flag column found a second bug — a hardcoded date range one week off from what the data actually showed. Both were fixed, the analysis was re-run, the dashboard was rebuilt from the corrected output, and the fix was verified live on the published site before this document was written.

Every number in this document, and on the live dashboard, reflects the corrected analysis — not the first version the pipeline produced.

What the analysis revealed

A network running with no functional reorder-point discipline (26 of 28 positions below target), a single-sourced supplier carrying disproportionate lead-time risk, and a supplier reliability field that couldn't be trusted at face value for any vendor in the portfolio — not just the obviously slow one.

How this demonstrates analyst skill

Not by writing the most code by hand. By not accepting generated output at face value, verifying it against source data, finding where it was wrong, being able to explain exactly why — then communicating the corrected findings clearly enough for a non-technical stakeholder to act on.

Key takeaway

A working, published data product with a documented, honest accuracy claim, a defensible inventory-planning methodology, and two concretely identified and fixed analytical bugs — the kind of scrutiny that matters more in a real analytics job than any single chart.

What This Project Actually Shows

Only what is genuinely demonstrated by the work above — not a generic tools list.

Core analytical skills

Data quality assurance

Cross-verification against source data

Root-cause debugging

Inventory & supply-chain analytics

Safety stock / reorder-point modeling

Forecast accuracy evaluation (WMAPE, holdout validation)

Business-focused data storytelling

Tools & working methods

Python (pandas, NumPy)

Directing AI-assisted analytics pipelines

Directing AI application builders (Lovable)

Git / GitHub project management

Technical writing for non-technical audiences

Evidence for each claim

SKILL	WHERE IT SHOWS UP IN THIS PROJECT
Data quality assurance	Censored-demand diagnosis and sibling-warehouse correction (page 07)
Root-cause debugging	Two pipeline bugs traced to exact lines and fixed (page 10)
Forecast evaluation	WMAPE on an 8-week blind holdout against a naive baseline (page 08)
Supply-chain modeling	Safety stock and reorder point with demand and lead-time variability (page 09)
Data storytelling	Published dashboard and the recommendation set on page 14

What this project does not claim. Hand-written SQL · a manually-built dashboard frontend · Power BI or Excel-based visualization — none of these were part of this project, so none are claimed here.

Where to Find Everything

The full analysis script, the corrected dataset output and the published dashboard are all publicly reviewable.

PROJECT REPOSITORY

View the project on GitHub →

github.com/tahtabirrahman/terratrak-insights

GITHUB PROFILE

View my GitHub profile →

github.com/tahtabirrahman

LIVE DASHBOARD

Open the published dashboard →

trac-planner-insight.lovable.app

RESOURCE	LOCATION
README	github.com/tahtabirrahman/terratrak-insights/blob/main/README.md
Analysis script	scripts/supply_chain_analysis.py
Jupyter notebook	Not provided — the analysis was run as a standalone script, not a notebook

Key takeaway. A published data product with an honest accuracy claim, a defensible planning methodology, and two analytical bugs found and fixed by checking the output against the source data.